

48730 Cyber Security - Week 3

DNS Pharming

48730 Cyber Security – Lab 2 (Week 3): DNS Pharming

Name: Jinkyung Kim

Student ID: 14657314

Date 19 August 2026

Task 1 – Attack by modifying the client's DNS setting (HOSTS / interfaces)

This first one is pretty straight forward out of the three. Essentially if you can get on the victim machine you can simply redirect the victims DNS resolver to point to a machine of your choosing. After this any name that is looked up will be answered by you. To do this for this task I modified the client machines /etc/network/interfaces file and changed the dns-nameservers line from the actual server (10.0.2.6) to the attacker (10.0.2.7).

Lets take a look at a “before” state. I ran a normal dig on the client so that I had something to compare it to later on. As you can see below it properly resolves www.netsec-week3.com to 10.0.2.101 and you can see at the bottom where it says SERVER: 10.0.2.6, this is the real DNS server.

```
cybersec-client@ubuntu:~$ dig www.netsec-week3.com

; <<>> DiG 9.9.5-3ubuntu0.8-Ubuntu <<>> www.netsec-week3.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 53086
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.netsec-week3.com.      IN      A

;; ANSWER SECTION:
www.netsec-week3.com.      259200  IN      A      10.0.2.101

;; AUTHORITY SECTION:
netsec-week3.com.          259200  IN      NS      ns.netsec-week3.com.

;; ADDITIONAL SECTION:
ns.netsec-week3.com.       259200  IN      A      10.0.2.10

;; Query time: 37 msec
;; SERVER: 10.0.2.6#53(10.0.2.6)
;; WHEN: Tue Aug 18 04:13:45 PDT 2026
;; MSG SIZE rcvd: 98

cybersec-client@ubuntu:~$
```

Figure 1 – dig before the change. Answer 10.0.2.101, SERVER: 10.0.2.6 (the real server).

So I ran `sudo nano /etc/network/interfaces` and edited the interfaces file, swapping the last octet on the nameserver line from 6 to 7. Minor detail but worth mentioning that I had to verify that the word was indeed `dns-nameservers` (with an s, plural) — for some reason `resolvconf` only looks for the plural version, so if you mistakenly leave off the s it just silently skips that line. Here's what the file looked like after the edit:

```
cybersec-client@ubuntu:~$ cat /etc/network/interfaces
# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo eth0
iface lo inet loopback

iface eth0 inet static
    address 10.0.2.8
    netmask 255.255.255.0
    gateway 10.0.2.1
    dns-nameservers 10.0.2.7

cybersec-client@ubuntu:~$
```

Figure 2 – `/etc/network/interfaces` after the edit, `dns-nameservers` now 10.0.2.7 (the attacker).

After rebooting (I ran `sudo ifdown eth0 && sudo ifup eth0` instead of rebooting. It's quicker and it recreates `resolv.conf` anyways), I did `dig` again. Notice how the `SERVER` line now says 10.0.2.7. This means that the client is forwarding all of its DNS queries to the attacker. That is exactly what this attack accomplishes — the victim is trusting a box that it shouldn't be trusting.

```
cybersec-client@ubuntu:~$ sudo ifdown eth0
cybersec-client@ubuntu:~$ sudo ifup eth0
cybersec-client@ubuntu:~$ dig www.netsec-week3.com

; <<>> DiG 9.9.5-3ubuntu0.8-Ubuntu <<>> www.netsec-week3.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 3418
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 2

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.netsec-week3.com.      IN      A

;; ANSWER SECTION:
www.netsec-week3.com.      259200  IN      A      10.0.2.101

;; AUTHORITY SECTION:
netsec-week3.com.         259200  IN      NS      ns.netsec-week3.com.

;; ADDITIONAL SECTION:
ns.netsec-week3.com.      259200  IN      A      10.0.2.10

;; Query time: 8 msec
;; SERVER: 10.0.2.7#53(10.0.2.7)
;; WHEN: Tue Aug 18 04:42:41 PDT 2026
;; MSG SIZE rcvd: 98

cybersec-client@ubuntu:~$
```

Figure 3 – dig after the change. SERVER: 10.0.2.7 now, i.e. the client is querying the attacker.

After I got the screenshot I put the nameserver back to 10.0.2.6, because Task 2 needs the client to be talking to the real server again to start with.

Task 2 – Attack by spoofing DNS responses on the network

Task 2 is the more interesting one. Instead of touching the victim's config, the attacker sits on the network, watches for DNS queries, and races to send back a forged answer before the real server's answer arrives. If the fake one lands first, the victim caches it and goes to the attacker's IP. I used Wireshark on the client to watch, and netwox tool 105 (“Sniff and send DNS answers”) on the attacker.

Baseline first. With Wireshark capturing on eth0 (filter dns), I ran a normal dig for `www.netsec-week3.com`. It comes back with 10.0.2.101 from server 10.0.2.6 like before.

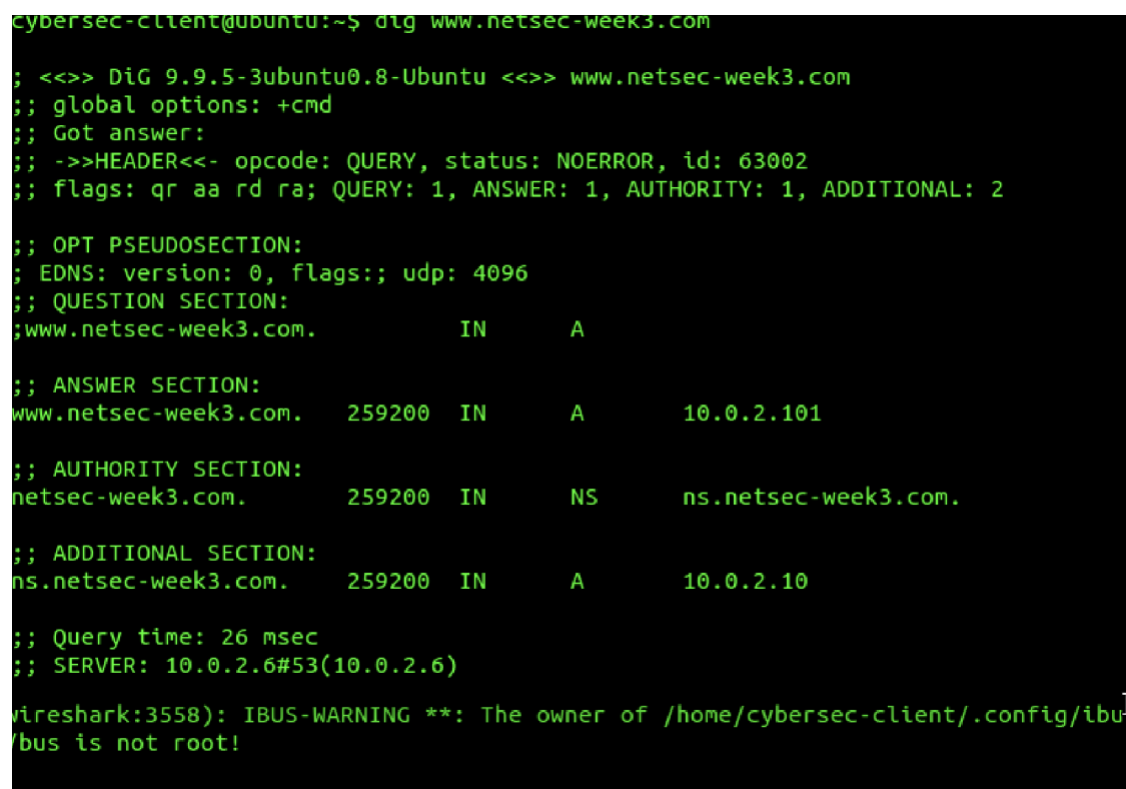


Figure 4 – baseline dig for Task 2 (before any spoofing). Still the legit 10.0.2.101 from 10.0.2.6.

In Wireshark I filtered down to just this name with `dns.qry.name == "www.netsec-week3.com"` so I could see the single query and its single response. This is the “normal” picture — one query out, one answer back.

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Figure 5 – the normal DNS exchange in Wireshark: one query (10.0.2.8 → 10.0.2.6) and one response.

Then on the attacker I set up netwox 105 through the netwag GUI. Hostname is the site I want to hijack, hostnameip is the fake address I want to send (10.0.2.7, the attacker itself), and I filled in the authns / authnsip fields too. Device had to be Eth0 — I actually got this wrong the first time and left it on the loopback (Lo0), and of course nothing happened because loopback never sees any of the LAN traffic. Took me a bit to notice that.

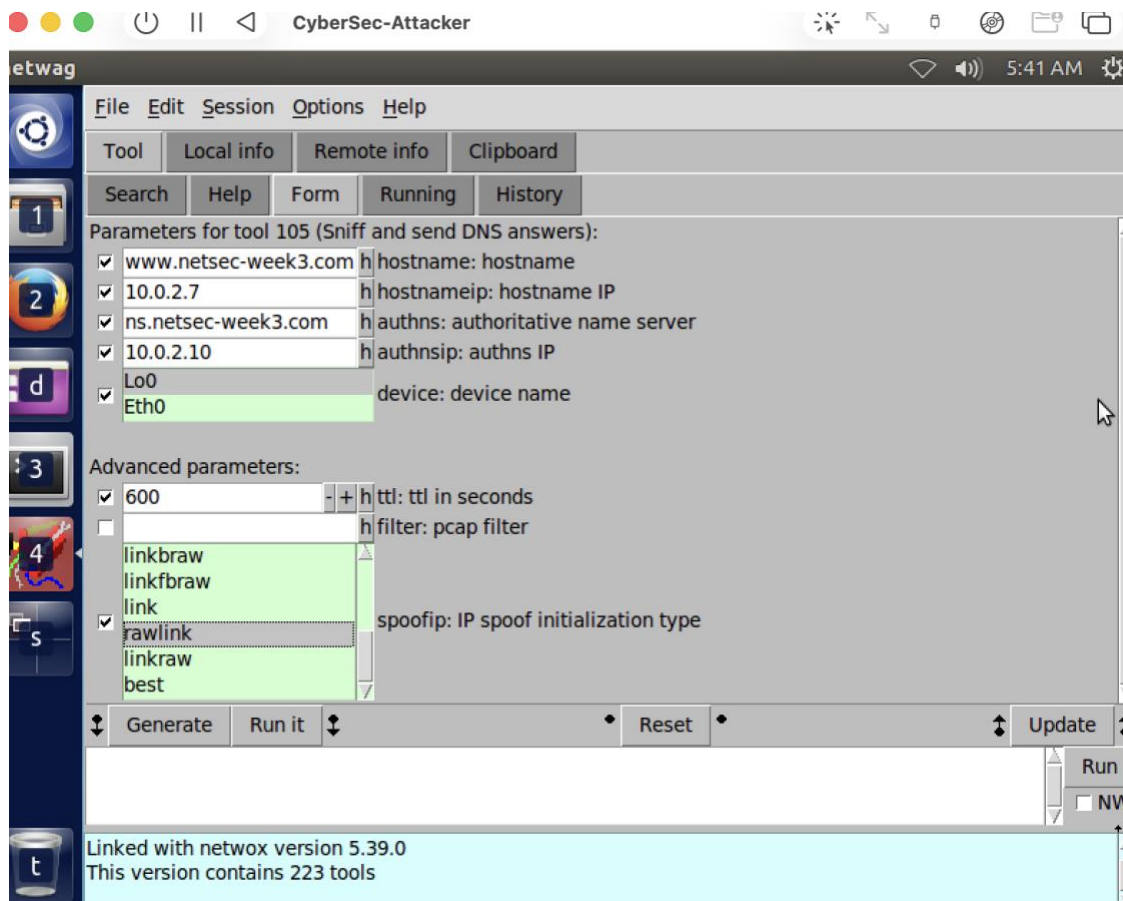


Figure 6 – netsec tool 105 configured to answer www.netsec-week3.com with the fake IP 10.0.2.7.

The part that didn't work at first (and why)

Here's where the Apple Silicon thing came back to bite me. I started tool 105, ran dig on the client a bunch of times, and... nothing. The client just kept getting the real 10.0.2.101. No forged answer showed up in Wireshark at all. I spent a while assuming I'd misconfigured something.

What was actually going on: I checked whether the attacker could even see the client's DNS query by running `tcpdump -i eth0 udp port 53` on the attacker while digging on the client, and it stayed completely silent. So the attacker was never seeing the query in the first place, which means netsec 105 had nothing to react to. The reason is that UTM's virtual network (Apple's vmnet) behaves like a proper switch, not a hub — it does not forward one VM's unicast traffic to another VM, and it doesn't seem to allow promiscuous sniffing across VMs. On VMware's internal network (which this lab was written for) you *can* turn promiscuous mode on and the sniff-and-spoof just works. On my setup it can't, so the standard version of the attack basically can't run as-is. That's a network backend limitation, not really the M2 chip itself, but the M2 is the reason I'm stuck on UTM/vmnet in the first place.

Getting it to actually work (ARP cache poisoning as a workaround)

To get past that I added an ARP poisoning step so the client's DNS query would physically come to the attacker instead of the real server. I used netwox tool 80 ("Periodically send ARP replies") to keep telling the client that 10.0.2.6 (the DNS server) lives at the attacker's MAC address (12:97:e6:72:a2:9c). Once that stuck, checking `arp -n` on the client showed 10.0.2.6 mapped to the attacker's MAC, which is exactly the lie I wanted. Now the client sends its DNS queries straight to the attacker, netwox 105 sees them, and answers them — and because the real server never even gets the query, the attacker basically wins by default instead of having to win a race.

So I had two tools running together on the attacker: tool 80 doing the ARP poisoning and tool 105 doing the DNS forging. After that, dig on the client finally returned the fake address. Below is the forged response captured in Wireshark — you can see the Answers section gives `www.netsec-week3.com` → 10.0.2.7, and in the raw bytes at the bottom the response's source MAC is the attacker's (12:97:e6:72:...), which is proof it didn't come from the real server.

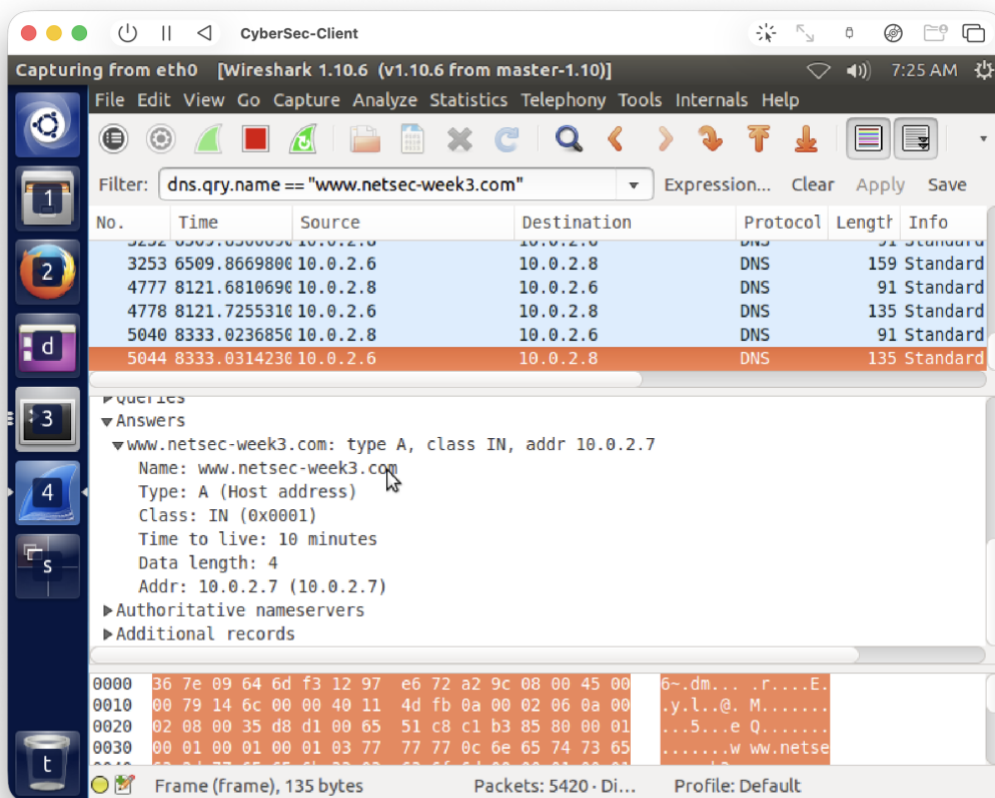


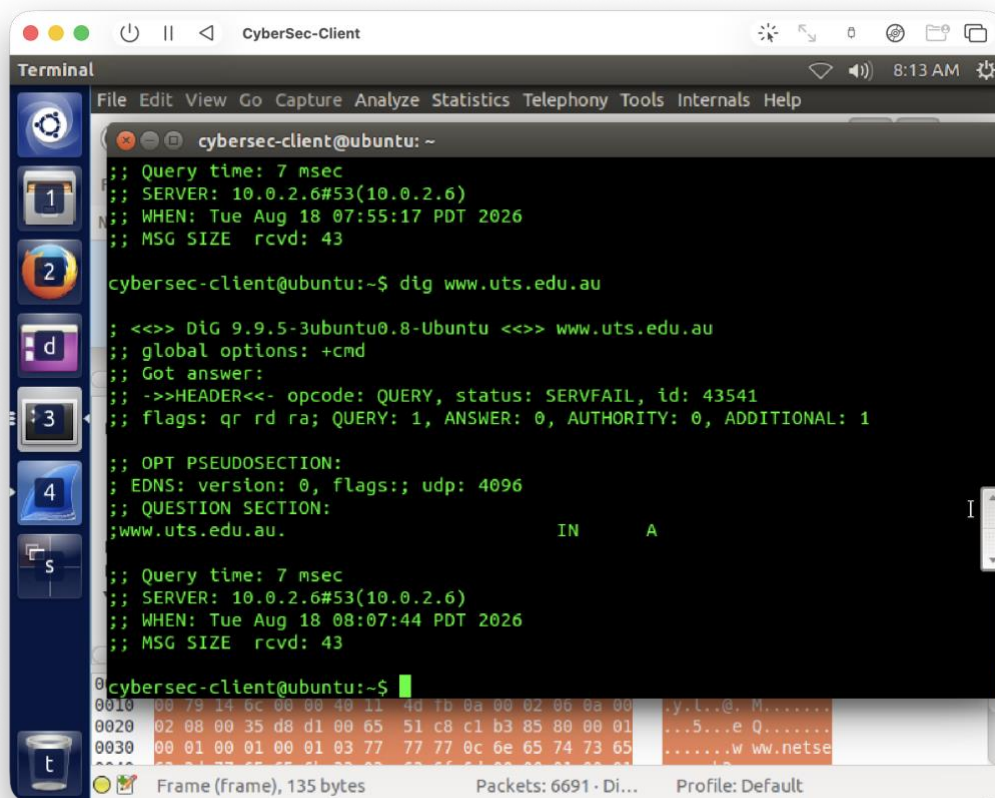
Figure 7 – the forged DNS response in Wireshark. Answer is 10.0.2.7 (attacker), TTL 10 min, and the Ethernet source is the attacker's MAC – so this reply is spoofed, not from the real 10.0.2.6.

Honestly this took longer than I expected (there was also a moment where I had a duplicate tool 105 running with the default demo values and it injected 1.2.3.4 instead of my 10.0.2.7, which confused me for a bit until I cleared the extra instances). But in the end the client resolves the site to the attacker's IP, which is the goal.

Task 3 – DNS server cache poisoning

The third task moves the target from the client to the server. The idea is: the client asks the server to resolve a name the server doesn't own (I used `www.uts.edu.au`), the server goes off to do a recursive lookup, and the attacker forges the answer coming back so the server stores the wrong record in its cache. Then every client that asks afterwards gets the poisoned answer, until the TTL runs out.

I flushed the server cache again first, and did a baseline dig from the client for `www.uts.edu.au`. Since the server has no internet it can't actually resolve it, so it just returns SERVFAIL, which is what I expected at this stage.



```
cybersec-client@ubuntu: ~  
;; Query time: 7 msec  
;; SERVER: 10.0.2.6#53(10.0.2.6)  
;; WHEN: Tue Aug 18 07:55:17 PDT 2026  
;; MSG SIZE rcvd: 43  
  
cybersec-client@ubuntu:~$ dig www.uts.edu.au  
  
;<><> DiG 9.9.5-3ubuntu0.8-Ubuntu <><> www.uts.edu.au  
;; global options: +cmd  
;; Got answer:  
;; ->HEADER<- opcode: QUERY, status: SERVFAIL, id: 43541  
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1  
  
;; OPT PSEUDOSECTION:  
;; EDNS: version: 0, flags:; udp: 4096  
;; QUESTION SECTION:  
;; www.uts.edu.au. IN A  
  
;; Query time: 7 msec  
;; SERVER: 10.0.2.6#53(10.0.2.6)  
;; WHEN: Tue Aug 18 08:07:44 PDT 2026  
;; MSG SIZE rcvd: 43  
  
cybersec-client@ubuntu:~$
```

0010 00 79 14 0c 00 00 40 11 40 1b 0a 00 02 00 0a 00 ...y.t...@. M.....
0020 02 08 00 35 d8 d1 00 65 51 c8 c1 b3 85 80 00 01 ...5...e Q.....
0030 00 01 00 01 00 01 03 77 77 77 0c 6e 65 74 73 65w ww.netse

Frame (frame), 135 bytes Packets: 6691 · Di... Profile: Default

Figure 8 – baseline for Task 3. Server can't reach the internet so `www.uts.edu.au` comes back SERVFAIL.

Same vmnet problem as Task 2 applies here, except worse, because now it's the *server's* recursive query I need the attacker to see, not the client's. So I re-pointed the ARP poisoning at the server this time — tool 80 telling the server that the gateway (10.0.2.1) is at the attacker's MAC, so the server's outbound recursive queries get funneled to the attacker. Then tool 105 set to forge answers for `www.uts.edu.au` pointing at 10.0.2.7.

The interception part definitely worked. In the netwag output you can see the attacker actually receiving and answering the server's recursive queries in real time (in the screenshot it's answering the server's lookups). Tool 80 and tool 105 are both running (bottom of the window), with tool 80 targeting the server's MAC 66:60:18:56:2b:9c. So the server's traffic really was being redirected to the attacker and the attacker really was feeding it forged responses.

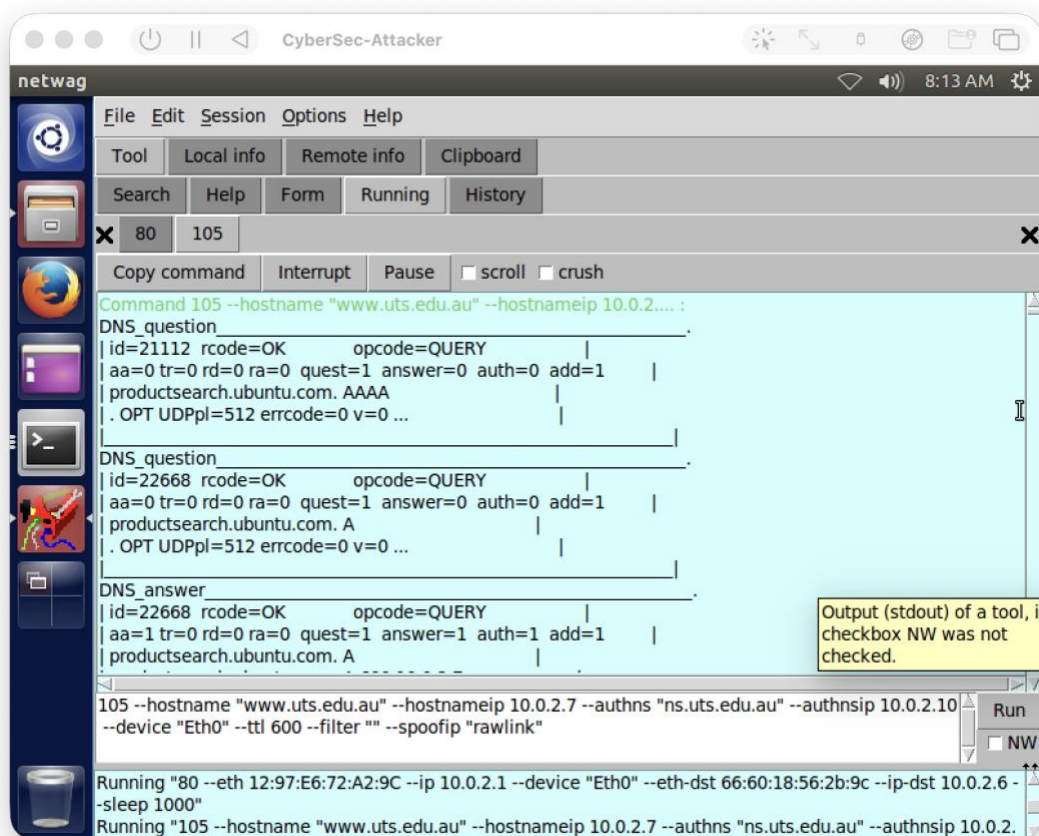


Figure 9 – the attacker intercepting and answering the server's recursive DNS queries (tool 80 ARP poisoning the server + tool 105 forging answers, both running).

What happened / why it didn't fully poison

Here's the honest result: even though the interception worked, the client's dig for `www.uts.edu.au` kept coming back SERVFAIL. The forged records never actually got into the server's cache. I'm fairly sure this is because the DNS server is running BIND 9.9.5, and modern BIND has a bunch of anti-spoofing protections — randomised source ports, checking that the answer comes from the exact server it queried, and validating the delegation chain during recursion. `netwox` can feed it forged answers all day but BIND just rejects the ones that don't line up, so nothing gets cached and the client keeps getting SERVFAIL. I did try pointing the fake delegation at the attacker to keep the chain going but it still didn't take.

So I'd summarise Task 3 like this: I managed the hard part (redirecting the server's recursive queries to the attacker and getting it to send forged answers — visible in Figure 9), but the actual cache poisoning was stopped by BIND's response validation. Which, thinking about it, is kind of the point — it shows that a properly patched resolver is actually pretty resistant to this attack, unlike the older resolvers the classic Kaminsky-style poisoning targeted. The intended “successful” outcome (client eventually getting `10.0.2.7` from the poisoned cache) is in the provided worked solution for the lab, but in my environment BIND's defences plus the `vmnet` restriction meant I could demonstrate the mechanism but not a full cache poison.

Reflection

Main thing I took away this week is that these DNS attacks are very dependent on the network being able to sniff other machines' traffic, which the emulated network on my Mac just doesn't allow by default. Working around that with ARP poisoning to force the traffic to the attacker was actually the most useful thing I learned — it made Task 2 work and got Task 3 to the interception stage. It also made it pretty concrete why the defences matter: switched networks (so you can't just sniff everyone), and resolvers that validate responses (so you can't just inject a fake answer), are exactly the things that stopped the attacks from fully landing. For a real network you'd also want DNSSEC and things like DHCP snooping / dynamic ARP inspection to shut down the ARP step I relied on.